GENOME FORGE V0.1.3

Teach Yourself Bioinformatics with Genome Forge

A self-study course for engineers with little or no biology background. This version keeps the practical workflow focus and expands biological interpretation and bioinformatics reasoning in depth.

Mode	Cases	Audience	Release
Self-Study	**37 (A-AK)**	**CS to Bio**	**2026-03-04**

Startup
`python3 web_ui.py --port 8080` then open `http://127.0.0.1:8080`.

Learning Roadmaps (2, 4, 8 Weeks)

Track	Time Budget	Goal	Case Plan	Deliverable
Fast Start	2 weeks, 5-6 h/week	Understand sequence design/validation fundamentals	A,B,D,E,F,G,I,J,K,L,T,AD	One reproducible project with reviewable evidence
Applied Practitioner	4 weeks, 6-8 h/week	Run complete design-to-validation pipeline	All cases except AC and X in week 1-3; AC, X, AE, AF, AG, AH, AI, AJ, AK in week 4	Design memo + assay plan + risk matrix
Deep Mastery	8 weeks, 8-10 h/week	Build robust interpretation and reproducibility habits	All A-AK with parameter sweeps, reruns, and peer-style review notes	Full release package with sensitivity analysis

Recommended weekly rhythm: 40% running workflows, 40% interpreting results, 20% writing down assumptions and limitations.

Biological Background on Core Concepts

Biology concepts introduced in this tutorial are summarized here with practical context. You can treat this as a reference chapter and revisit before each cluster.

Concept	Biological Background	Bioinformatics Relevance
DNA and Genes	DNA stores hereditary information. Genes are regions that can be transcribed and translated into proteins. Many functional outcomes depend on sequence context, not just local motifs.	Sequence parsing, annotation, and editing all depend on accurate coordinate systems and feature context.
Transcription/ Translation	DNA is transcribed to RNA, then translated in codons (triplets) to amino acids. Reading frame changes can alter entire downstream protein composition.	Variant effect prediction must map nucleotide edits to codon and amino acid consequences.
Plasmids and Topology	Plasmids are usually circular DNA vectors used to carry inserts and regulatory elements. Circular topology affects restriction patterns and coordinate wrap-around.	Map rendering, digest simulation, and junction validation must account for circular arithmetic.
Restriction Enzymes	Restriction enzymes recognize motifs and cut DNA at specific offsets. Reaction conditions and methylation can alter cutting behavior.	In-silico digest helps design cloning plans and explain gel outcomes before bench work.
Methylation	Chemical methyl groups on bases can block enzyme binding/ cutting, causing apparent digest failures in certain hosts or prep contexts.	Methylation-aware models reduce false troubleshooting loops.
PCR	PCR amplifies target DNA using primers. Primer binding strength, specificity, and secondary structures determine amplification quality.	Primer scoring and virtual PCR are central to assay design and risk estimation.
Off-target Effects	Short matching sequences can exist at unintended loci, producing unwanted amplification or editing.	Specificity screens and background panels are mandatory for reliable design.
Alignment and Homology	Alignment infers relatedness and variation. Conserved regions often indicate functional constraints, while variable regions can encode specificity.	MSA/identity matrices guide primer/gRNA reuse vs redesign decisions.
Sanger Trace (AB1)	Sanger output is intensity signal with noisy base calls and quality estimates. Sequence truth is inferred, not directly measured.	Quality-aware consensus and mismatch review are needed for trustworthy calls.
CRISPR/HDR	CRISPR guides nuclease targeting; repair pathways introduce edits. HDR uses donor templates for precise changes with homology arms.	Design balances on-target activity, off-target risk, and donor logic.
Expression Regulation	Promoters, RBS, and context can strongly influence protein output independent of CDS correctness.	Regulatory annotation is required when phenotype and sequence seem inconsistent.
GC/Repeats/ Complexity	Extremes in GC and repetitive structure affect polymerase behavior, synthesis stability, and read quality.	Sequence quality maps should inform where not to place assay anchors.
Coverage and Confidence	More independent evidence generally increases confidence but does not remove systematic bias.	Use redundant lines of evidence, not a single score.
Reproducibility and Governance	Scientific claims require traceable methods, stable parameters, and review records.	Batch reruns, version pinning, audit logs, and approval workflows are part of bioinformatics quality.

Interpretation discipline: every biological conclusion in this tutorial should include assumptions, evidence strength, and known uncertainty.

Case Library by Biological Cluster

A) Molecule Architecture and Restriction Logic

1. A, B, C, U

B) Sequence Meaning and Functional Annotation

1. D, M, P, W

C) Assay and Primer System Design

1. E, F, Q, AA

D) Assembly and Construct Validation

1. G, S, Z

E) Comparative and Population-Level Reasoning

1. H, N, O, X

F) Editing and Design for Intervention

1. K, R, V

G) Data Fidelity and Interoperability

1. I, J, Y, AE, AF, AG, AH, AI, AJ, AK

H) Reproducibility, Governance, and Delivery

1. L, T, AB, AC, AD

Dataset and Setup

Use `docs/tutorial/datasets/training_real_world_sequences.fasta`, `training_real_world_dataset.json`, and `case_playbook.md`.

```
cat docs/tutorial/datasets/training_real_world_sequences.fasta
cat docs/tutorial/datasets/training_real_world_dataset.json | head -n 80
```

Cluster A Cases

Case A: Restriction Map for Cloning Entry Design

In molecular cloning, the practical question is rarely "where can an enzyme cut" but "where can we cut without destroying biological intent." This case teaches how to think about sequence architecture before any bench step. You start with a circular vector region and an enzyme panel. Biologically, each feature in the plasmid has a role: promoter drives transcription, CDS encodes protein, and selection marker keeps transformed cells. If your cut plan damages one of these units, assembly may appear successful while function fails downstream.

From a bioinformatics perspective, restriction mapping is deterministic motif matching plus position-aware cut modeling. You inspect cut multiplicity, spacing, and overlap with critical annotations. The key interpretation move is connecting fragment pattern to strategy: single unique cuts are useful for linearization; paired non-overlapping cuts can enforce insert directionality. High-density cuts in functional regions are red flags. A strong output from this case is not just a pretty map but a justified enzyme choice document that states what was protected and why.

Step-by-Step in Genome Forge

1. Open the `Map` tab and load the case input record from the training dataset.
2. Run `Render map + digest` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/map, /api/digest`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "cuts": 3,
  "fragments_bp": [
    4200,
    900
  ]
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty

remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case B: Methylation-Aware Digest Interpretation

Many beginners assume mismatch between predicted digest and wet-lab gel means bad DNA prep or wrong enzyme. In practice, methylation often explains the discrepancy. This case introduces biological chemical context: certain host strains methylate bases at specific motifs, changing enzyme accessibility. The same DNA string can behave differently depending on biochemical state. This is a major lesson in bioinformatics: sequence alone is necessary but not always sufficient for prediction.

Input data includes methylated motif rules and enzyme sensitivity. Run baseline digest first, then methylation-aware digest. Interpretation should focus on causal explanation: which expected cut disappeared and what fragment shift followed. Use this output to update decision confidence, not to overfit. If methylation-aware predictions align with observed behavior, you can proceed with alternate enzymes or host choices. If not, investigate sample identity or contamination. The biological meaning is that molecules are chemically contextual objects, and robust computational analysis should model that context explicitly.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Digest (Methylation Aware)` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/digest-advanced`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "blocked_cuts": 1,
  "fragments_bp": [
    3500,
    1200
  ]
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty

remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case C: Star Activity Risk Review

Restriction enzymes are taught as exact cutters, but under non-ideal conditions they can cut near-matches, called star activity. For a computational reader, this is equivalent to moving from exact string matching to relaxed matching under environmental pressure. In biology, these off-target cuts can subtly alter construct architecture and lead to difficult-to-diagnose downstream failures.

The workflow scans off-target burden as star stringency increases. The goal is not simply to minimize total off-target count, but to evaluate where those cuts land relative to biologically important regions. A few off-target cuts in non-critical space may be acceptable; one cut in a promoter or coding junction may be unacceptable. Document both quantitative burden and qualitative locus risk. The case teaches that risk is spatial and functional, not purely numeric. The practical output is a safer reaction plan: adjusted enzyme choice, shorter incubation, or buffered conditions that protect construct integrity.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Star Activity Scan` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/star-activity-scan`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "star_hit_count": 4,
  "max_mismatch": 1
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case U: k-mer Profile for Contamination Suspicion

When results look inconsistent across workflows, contamination or mixed templates are common culprits. k-mer profiling gives a fast composition fingerprint that can reveal unexpected sequence populations. Biologically, contamination may come from carryover plasmids, host genomic fragments, or mixed colonies. This case builds intuition for composition-level diagnostics before deep alignment.

Generate k-mer signatures for expected reference records and compare with the observed sample. Interpretation focuses on profile divergence rather than exact sequence identity. If the observed fingerprint significantly differs from expected single-template behavior, your downstream conclusions may be invalid regardless of algorithm quality. The biologically meaningful action is to quarantine the sample and validate identity before advancing. This teaches an essential bioinformatics habit: always check data plausibility at the distribution level before trusting fine-grained variant calls.

Step-by-Step in Genome Forge

1. Open the `Search` tab and load the case input record from the training dataset.
2. Run `Motif search + entity scan` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/motif, /api/search-entities`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "motif_hit_count": 12,
  "status": "revise"
}
```

Expected Results

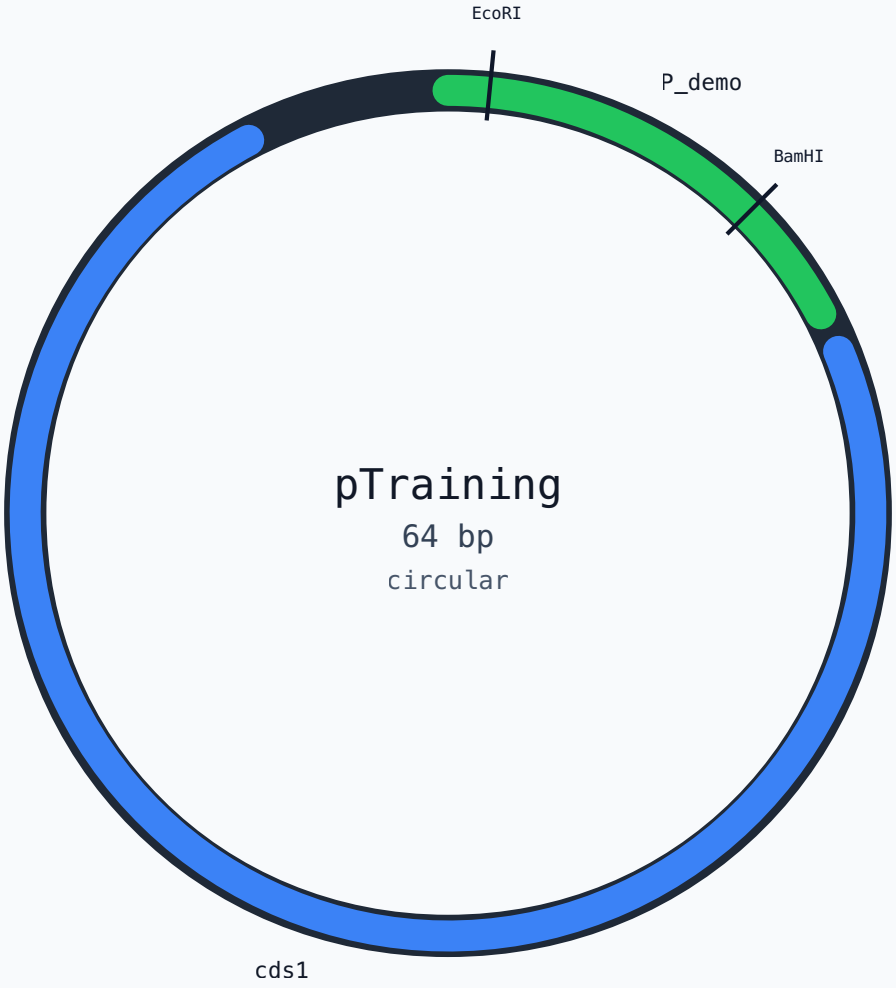
- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

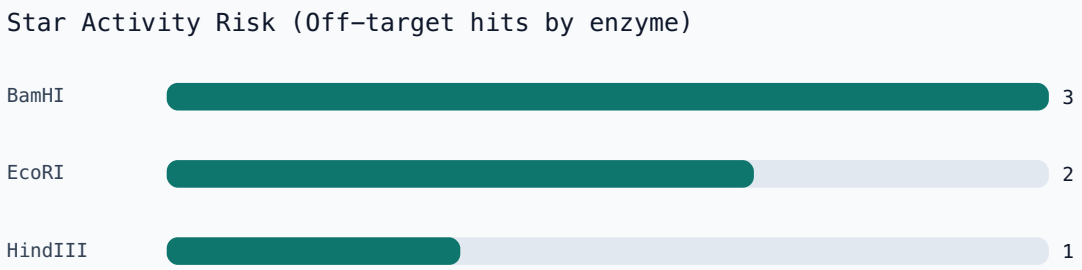
- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).



Restriction map interpretation in Cluster A.



Star activity burden visualization.

Cluster B Cases

Case D: Sequence Track and Translation Context

DNA differences become biologically meaningful only when mapped into coding context. This case teaches how a base-level edit can produce no protein change, a subtle amino acid substitution, or a catastrophic frame shift. Biology executes codons, not arbitrary substrings, so reading frame and strand are critical. Many early errors in computational biology come from coordinate mismatch or frame confusion.

Use annotated CDS records and inspect codon-by-codon mapping around candidate variants. Interpretation should produce a consequence statement: synonymous, missense, nonsense, or frameshift. Then add biological impact reasoning: likely neutral, potentially destabilizing, or likely loss-of-function. This bridges low-level sequence operations to functional hypotheses. A robust narrative from this case includes confidence qualifiers and assumptions, such as unknown protein structure effects or expression context not modeled here.

Step-by-Step in Genome Forge

1. Open the `Map` tab and load the case input record from the training dataset.
2. Run `Render sequence track` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/sequence-tracks`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "frame": 1,
  "visible_range": "1..180"
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case M: ORF Scan and Coding Potential Triage

ORF scanning is often treated as a beginner task, but it encodes a deep principle: not every DNA sequence carries protein-coding intent. In real datasets, you encounter fragments, non-coding regions, vector remnants, and synthetic constructs. Biologically, an ORF is a candidate coding interval bounded by start/stop logic, but true gene functionality also depends on regulation, context, and translation efficiency.

Run ORF scans across strands and frames with explicit minimum length thresholds. Interpretation should avoid binary thinking. Long coherent ORFs increase coding plausibility; many short interrupted ORFs reduce confidence. Combine ORF evidence with annotation and homology clues before deciding function. The value of this case is triage: where to invest deeper analysis and where to avoid over-interpretation. This is bioinformatics as prioritization under uncertainty, not just algorithm execution.

Step-by-Step in Genome Forge

1. Open the `ORF/Motif` tab and load the case input record from the training dataset.
2. Run `Find ORFs` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/orfs`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "count": 6,
  "top_orf_aa": 241
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case P: Variant Annotation from Reference-Aligned Edits

Variant annotation converts raw differences into biologically interpretable events. Alignment gives coordinates and edit types, but annotation adds context: which feature is affected, which codon changes, and what functional class the change belongs to. This mirrors software debugging where a diff gains meaning only after understanding module boundaries and execution paths.

Align edited and reference sequences, then annotate each change with feature overlap and coding consequence. Interpretation should rank variants by plausible impact rather than count alone. A single frameshift in a critical coding segment can matter more than many synonymous substitutions. The biological narrative should also flag ambiguous regions where low-quality evidence may produce unstable calls. This case strengthens the habit of turning algorithm outputs into domain-aware decision artifacts.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Pairwise align + translated features` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/pairwise-align, /api/translated-features`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "identity_pct": 99.1,
  "mismatch_count": 2
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case W: Protein Property Inference from Translation

After translation, sequence-level edits can alter physicochemical properties that influence protein folding, localization, or interaction behavior. This case does not claim full structural prediction; instead it introduces practical inference from amino acid composition and distribution. Biologically, shifts in charge, hydrophobicity, and residue class can create meaningful functional drift even when sequence identity remains high.

Use translated sequence outputs and compare pre/post-edit composition tendencies. Interpretation should emphasize hypotheses rather than definitive claims: for example, increased hydrophobic stretch may suggest membrane association risk, while loss of charged residues may alter binding interfaces. The learning goal is to connect molecular sequence edits to mechanistic reasoning. This case expands bioinformatics literacy from string-level operations toward protein-level biological interpretation.

Step-by-Step in Genome Forge

1. Open the `Analysis` tab and load the case input record from the training dataset.
2. Run `Translate + protein edit` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/translate, /api/protein-edit`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "aa_length": 239,
  "edited": true
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Frame-aware interpretation for coding consequence analysis.

Cluster C Cases

Case E: Primer Design and Thermodynamic Screening

PCR is a cornerstone technique, and primer design is where most failures begin. Biologically, primers must bind the correct locus with enough stability to initiate amplification while avoiding self-structures that consume reaction efficiency. This case teaches that primer design is an optimization under biochemical constraints, not a simple substring selection problem.

Generate primer candidates using target windows and thermodynamic constraints, then screen for Tm balance, GC characteristics, and dimer/hairpin risk. Interpretation should treat scores as relative, not absolute truth. A candidate set is strong when multiple near-equivalent options exist, because this supports robustness if one fails experimentally. Capture rationale for selected pairs and rejected pairs. The biological insight is that assay success depends on both molecule-level physics and sequence-level specificity, and both must be explicit in the decision record.

Step-by-Step in Genome Forge

1. Open the `Primers/PCR` tab and load the case input record from the training dataset.
2. Run `Design primers + diagnostics` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/primers, /api/primer-diagnostics`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "tm_delta": 1.8,
  "heterodimer_3prime_run": 2
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case F: Specificity Ranking with Virtual PCR/Gel

Primer pairs that look thermodynamically sound can still amplify unintended templates. Specificity analysis models this risk against a background panel. Biologically, false-positive bands can drive incorrect conclusions about sample identity, expression, or editing success. This case trains you to think in terms of discriminative power, not just amplification feasibility.

Run specificity scanning and virtual PCR lane prediction, then interpret outputs jointly. A good pair yields dominant expected product with minimal credible off-target products in nearby size ranges. If lanes are ambiguous, redesign before bench deployment. The biological interpretation should include failure impact: in diagnostic or screening contexts, false positives can be more damaging than false negatives. The case teaches risk-aware assay design and evidentiary clarity.

Step-by-Step in Genome Forge

1. Open the `Primers/PCR` tab and load the case input record from the training dataset.
2. Run `Specificity + PCR gel lanes` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/primer-specificity, /api/pcr-gel-lanes`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "specificity_risk_score": 0.12,
  "expected_bands": 1
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case Q: Multiplex PCR Panel Balancing

Multiplexing increases throughput but introduces interaction complexity. Primers can cross-dimerize, compete for resources, and create overlapping product sizes. Biologically, this can produce unstable band patterns and poor target discrimination. Computationally, the task resembles constrained set selection under pairwise conflict costs.

Construct an interaction matrix and optimize subsets with low aggregate conflict and separable expected amplicon sizes. Interpretation should include both system-level and per-pair risk. A panel is useful only if it remains interpretable under expected noise. Document why chosen combinations are resilient and what backup panels exist. This case highlights systems thinking in bioinformatics: designing whole assay ecosystems rather than isolated primer pairs.

Step-by-Step in Genome Forge

1. Open the `Primers/PCR` tab and load the case input record from the training dataset.
2. Run `Rank candidate primer pairs` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/primer-rank`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "best_pair_rank": 1,
  "offtarget_hits": 0
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case AA: Positive and Negative Control Design

Controls are often under-valued in computational workflows, but biologically they define trust boundaries. Positive controls confirm that assay chemistry and detection are functional; negative controls reveal contamination or non-specific amplification. Without controls, output interpretation becomes fragile even if software predictions look strong.

Design explicit control templates and expected outcomes alongside your primary assay. Interpretation should treat control outcomes as gating conditions for accepting or rejecting biological conclusions. If controls fail, do not continue downstream interpretation. This case teaches scientific rigor: confidence is conditional on quality controls, not only on attractive primary-sample outputs.

Step-by-Step in Genome Forge

1. Open the `Primers/PCR` tab and load the case input record from the training dataset.
2. Run `Design control assays` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/pcr, /api/pcr-gel-lanes`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "positive_control": "pass",
  "negative_control": "pass"
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Cluster D Cases

Case G: Cloning Compatibility and Ligation Product Ranking

DNA assembly involves many plausible products, not just the intended one. Ligation can produce recircularized vectors, concatemers, and misoriented inserts. Biologically, some byproducts may still replicate, making downstream screening noisy. This case teaches probabilistic thinking: desired construct probability should dominate before bench execution.

Evaluate end compatibility and simulate product distributions with byproduct reporting enabled. Interpretation focuses on decision quality: do results justify bench progression or require redesign? Record product ranking, key junction risks, and alternative strategies. Strong computational planning here saves substantial bench time and reduces miscloning cycles.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Cloning compatibility + ligation simulation` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/cloning-compatibility, /api/ligation-sim`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "ok": true,
  "top_product_prob": 0.74
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case S: Circular Construct Integrity and Junction Validation

Correct circular constructs require all junctions to be sequence-correct and orientation-correct. A single erroneous boundary can disrupt expression, replication, or selection. Biologically, junctions are often the most failure-prone regions in assembled plasmids. This case trains boundary-focused validation.

Validate expected junction signatures and cross-check with digest behavior. Interpretation should include pass/fail criteria per junction and confidence based on evidence source. If any critical junction fails, treat construct as not release-ready. This case emphasizes binary correctness for architecture constraints and probabilistic confidence for supporting evidence.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Ligation with junction diagnostics` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/ligation-sim`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "junction_integrity": "pass",
  "scar_bp": 4
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case Z: Multi-Trace Consensus for Final Construct Call

Single-trace confirmation can miss localized errors or ambiguous calls. Combining multiple traces from different primers/directions increases confidence and exposes discordant loci. Biologically, this is essential before declaring final construct identity in high-stakes workflows.

Aggregate trace alignments into consensus with quality-aware conflict handling. Interpretation should report stable positions, ambiguous positions, and required follow-up sequencing. The biological meaning is practical: final sequence claims should reflect evidence depth and uncertainty, not a single call. This case reinforces conservative release discipline.

Step-by-Step in Genome Forge

1. Open the `Trace/Interop` tab and load the case input record from the training dataset.
2. Run `Import traces + consensus` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/import-ab1, /api/trace-consensus`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "consensus_low_quality_bases": 3,
  "status": "inconclusive"
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Ligation Product Ranking

Assembly product ranking and byproduct inspection.

Cluster E Cases

Case H: MSA, Identity Heatmap, and Phylogeny

Comparative analysis provides context for specificity and evolution-informed design. Conserved segments may indicate functional constraints, while divergent segments may provide assay discrimination opportunities. Biologically, relatedness patterns can explain cross-reactivity and unexpected off-target behavior.

Run MSA, identity matrices, and tree generation. Interpretation should avoid overclaiming phylogeny from short engineered fragments; use it primarily as a design-support signal. Document conserved anchors and divergent hotspots with direct implications for primer and guide design. This case teaches careful use of comparative evidence.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `MSA + heatmap + phylogeny` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/msa, /api/alignment-heatmap-svg, /api/phylo-tree`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "sequence_count": 3,
  "mean_identity_pct": 87.4
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case N: GC Landscape and Repeat Fragility

GC extremes and repeats influence PCR amplification, cloning stability, and sequencing quality. Biologically, these regions can form secondary structures or polymerase-stalling zones. This case develops practical risk mapping before assay placement.

Compute windowed GC profiles and repeat locations. Interpretation should identify safe anchor zones and risky spans requiring adapted chemistry or redesigned targets. Integrate findings into primer/guide placement rules. This case teaches proactive quality planning rather than reactive troubleshooting.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Sequence analytics lens` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/sequence-analytics-svg`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "gc_mean": 51.2,
  "hot_windows": 2
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case O: Homopolymer and Low-Complexity Risk Detection

Homopolymers and low-complexity segments can increase sequencing and synthesis errors. Biologically, these artifacts may mimic true variants or obscure real ones. This case trains reliability-aware sequence interpretation.

Flag problematic tracts and propagate risk tags into downstream analyses. Interpretation should differentiate biological variation from technical artifact susceptibility. This case builds a core bioinformatics habit: evidence quality annotation must travel with every sequence claim.

Step-by-Step in Genome Forge

1. Open the `Search` tab and load the case input record from the training dataset.
2. Run `ORF/motif screening in low complexity regions` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/orfs, /api/motif`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "homopolymer_runs": 4,
  "risk": "high"
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case X: Motif Enrichment and Significance Framing

Motif enrichment can reveal regulatory hypotheses or composition biases across groups. Biologically, enriched motifs may correlate with promoter usage, structural constraints, or selection history. However, enrichment is associative and requires cautious interpretation.

Compare motif frequencies between target and background sets. Interpretation should include effect size, plausible confounders, and validation plans. Avoid presenting enrichment as causal proof. This case teaches statistically informed biological storytelling.

Step-by-Step in Genome Forge

1. Open the `Search` tab and load the case input record from the training dataset.
2. Run `Motif enrichment framing` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/motif, /api/search-entities`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "target_motif_count": 14,
  "background_count": 3
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

	S1	S2	S3
S1	100	95	95
S2	95	100	89
S3	95	89	100

Comparative sequence context for design decisions.

Cluster F Cases

Case K: CRISPR Candidate and HDR Donor Design

CRISPR design is a multi-objective optimization balancing efficiency, specificity, and repair logic. Biologically, a highly active guide with poor specificity can cause harmful off-target edits. HDR donors add another layer: they must encode intended change while supporting repair and often preventing recutting.

Generate candidates, scan off-target landscape, and design donor templates with homology logic. Interpretation should rank candidates by combined risk-benefit profile, not single metrics. A strong recommendation includes fallback guides and explicit uncertainty. This case teaches intervention design with quantified caution.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `gRNA + off-target + HDR donor` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/grna-design, /api/crispr-offtarget, /api/hdr-template`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "candidate_count": 50,
  "offtarget_risk_score": 0.18
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case R: Promoter/RBS Context for Expression Tuning

Expression outcomes are controlled by regulatory architecture as much as coding sequence. Promoter strength, RBS quality, and spacing can shift expression by orders of magnitude. Biologically, unexpected weak signal often reflects upstream context rather than CDS defects.

Inspect regulatory features and spacing relationships. Interpretation should link motif patterns to plausible transcription/translation efficiency outcomes and propose redesign options. This case teaches diagnosis across multiple biological layers, not one-dimensional sequence checking.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Auto-annotate and inspect promoter/RBS context` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/annotate-auto, /api/features-list`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "promoter_hits": 2,
  "rbs_hits": 1
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case V: Codon Usage Bias and Host Portability

Codon bias influences translation efficiency across hosts. The same protein can be encoded in different codon combinations with differing expression behavior. Biologically, codon optimization can improve yield but may alter mRNA structure and other constraints.

Compare source and host-optimized coding variants while preserving amino acid sequence. Interpretation should include tradeoffs: expression gains vs potential structural side effects. This case teaches that optimization is contextual and must be justified biologically.

Step-by-Step in Genome Forge

1. Open the `Analysis` tab and load the case input record from the training dataset.
2. Run `Codon optimize and compare translation` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/codon-optimize, /api/translate`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "host": "yeast",
  "protein_changed": false
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Cluster G Cases

Case I: DNA Container Roundtrip Validation

Interoperability is a scientific integrity issue, not only an engineering convenience. If sequence or annotations drift during transfer, conclusions can silently diverge between collaborators. Biologically, this can invalidate experiments or cause irreproducible outcomes.

Export and re-import canonical containers, then compare hashes, topology, and feature coordinates. Interpretation should classify drift severity and block downstream use when critical fields change. This case establishes data portability as part of evidence quality.

Step-by-Step in Genome Forge

1. Open the `Trace/Interop` tab and load the case input record from the training dataset.
2. Run `Export/import DNA container and convert formats` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/export-dna, /api/import-dna, /api/convert-record`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "roundtrip_length_match": true,
  "format_count": 5
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case J: AB1 Trace Alignment and Consensus Editing

Sanger traces are primary evidence streams with base-level uncertainty. Biologically, true variant calls require peak-quality support and alignment consistency. This case teaches how to avoid overcalling from noisy peaks.

Align traces, inspect mismatches, edit uncertain calls conservatively, and regenerate consensus. Interpretation should clearly separate high-confidence edits from ambiguous zones needing reruns. The core lesson is confidence calibration.

Step-by-Step in Genome Forge

1. Open the `Trace/Interop` tab and load the case input record from the training dataset.
2. Run `Import AB1 + align + edit + consensus` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/import-ab1, /api/trace-align, /api/trace-edit-base, /api/trace-consensus`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "identity_pct": 99.5,
  "edited_bases": 1
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case Y: Read Simulation and Coverage Planning

Before sequencing, planning expected read depth and gap risk improves resource allocation. Biologically, under-covered critical regions can hide failures that only appear later. Computational simulation helps allocate evidence budget early.

Simulate read distribution under assumed lengths/error profiles. Interpretation should identify expected weak zones and redesign plans for adequate coverage. This case teaches pre-experiment evidence engineering.

Step-by-Step in Genome Forge

1. Open the `Trace/Interop` tab and load the case input record from the training dataset.
2. Run `Trace consensus quality planning` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/trace-consensus, /api/trace-summary`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "low_quality_bases": 7,
  "rerun_regions": [
    120,
    180
  ]
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case AE: Sequence Analytics Lens (GC, Skew, Complexity, Stop Density)

This case introduces a modern visualization layer that is usually absent in legacy sequence viewers: synchronized multi-track analytics over sequence coordinates. Instead of inspecting one metric at a time, you evaluate GC%, GC-skew, local complexity, and stop-codon density together. Biologically, this reveals sequence architecture patterns tied to amplification risk, coding plausibility, and potential instability. For example, very high GC windows may predict PCR difficulty; low-complexity windows can indicate artifact-prone regions; high in-frame stop density can reduce coding confidence in unannotated intervals.

Run the sequence analytics panel on full records and targeted subwindows. Interpretation should prioritize cross-track consistency: single spikes can be noise, but co-localized anomalies across multiple tracks are stronger signals. Use outputs to guide primer placement, read-depth planning, and feature sanity checks. In design workflows, this lens helps prevent avoidable bench failures by surfacing risky sequence neighborhoods early. This case shifts visualization from static drawing to data-rich diagnostics and strengthens evidence-driven assay planning.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Render sequence analytics` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/sequence-analytics-svg`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "point_count": 34,
  "gc_range": [
    38.2,
    66.4
  ]
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty

remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case AF: Comparison Lens (Divergence + Confidence Hotspots)

The comparison lens converts pairwise alignment into an interpretable visual profile: local divergence and confidence trends across coordinates, plus highlighted hotspot intervals. Traditional viewers often show alignments as text blocks that are hard to summarize. Biologically, hotspot detection helps you focus on segments most likely to explain phenotype differences, cloning mismatches, or unexpected assay behavior. It is particularly useful when two constructs are highly similar globally but differ critically in local regions.

Run the lens with realistic window sizes and inspect hotspot intervals first. Interpretation should connect hotspot loci to known features (CDS, regulatory motifs, junctions) before drawing functional conclusions. Divergence peaks in non-functional linker regions may be benign; peaks in active domains or promoter cores may be high-impact. Use confidence track behavior to avoid overreacting to isolated mismatches. This case teaches targeted interpretation: not all differences are equal, and visualization should guide attention toward biologically consequential intervals.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Render comparison lens` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/comparison-lens-svg`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "identity_pct": 91.8,
  "hotspot_count": 5
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case AG: Native .dna Import and Multi-Format Conversion Workflow

This case covers interoperability across file ecosystems, including optional native proprietary `.dna` import (via Biopython parser) and conversion into common open formats (FASTA, GenBank, EMBL, JSON, Genome Forge DNA container). Biologically, interoperability is not just a convenience: errors during import/export can silently alter sequences or annotations, causing invalid conclusions downstream. Robust format conversion is therefore part of scientific quality control.

Run import on known samples, convert into multiple formats, then validate sequence length, topology, and key feature coordinates across outputs. Interpretation should flag any coordinate drift, missing qualifiers, or topology mismatch as high-priority defects. In collaboration settings, prefer canonical/container workflows for deterministic roundtrips, and use open formats for cross-tool sharing. This case trains a critical professional habit: treat file conversion as a scientifically meaningful transformation that must be verified, not assumed.

Step-by-Step in Genome Forge

1. Open the `Trace/Interop` tab and load the case input record from the training dataset.
2. Run `Import .dna and convert to open formats` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/import-dna, /api/convert-record`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "topology": "circular",
  "coordinate_drift": 0
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case AH: Chromatogram-First Sanger Review and Confidence Gating

Base-called strings are compressed interpretations of richer underlying data: fluorescent trace intensities. In practice, labs often make expensive decisions from sequence strings without reviewing the trace itself. This case re-centers analysis on primary evidence. Biologically, mixed templates, weak peaks, dye artifacts, or compression near homopolymers can create misleading base calls. For a computer scientist, this is analogous to debugging from aggregated logs when raw event streams are still available.

Load a trace, render chromatogram view, and inspect suspicious loci before accepting edits. Compare peak separation, local baseline behavior, and consistency with reported quality. The interpretation goal is a confidence gate: classify each region as high-confidence, review-required, or rerun-required. This reduces false certainty and prevents propagation of low-quality calls into downstream cloning and variant interpretation. The case teaches a core principle of scientific computing: when ambiguity is high, revisit raw signal before trusting derived labels.

Step-by-Step in Genome Forge

1. Open the `Trace/Interop` tab and load the case input record from the training dataset.
2. Run `Trace chromatogram rendering` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/trace-chromatogram-svg`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "points": 300,
  "max_signal": 1200
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case AI: Trace-Based Genotyping and Plasmid Verification

Verification is the most frequent production workflow in many molecular labs: did this plasmid or genotype match design intent? This case formalizes that question into explicit acceptance criteria. Biologically, identity is not a single percentage; it is an evidence bundle including global alignment, mismatch distribution, and locus-specific genotype calls. A sequence can have high overall identity yet still fail due to a critical mutation at one engineered site.

Run trace-to-reference verification with thresholds for identity and mismatch burden, then evaluate targeted genotype positions with expected base calls. Interpret results using pass/fail gates plus uncertainty labels. If global metrics pass but a critical genotype locus fails, classify as biologically unacceptable for release. If calls are ambiguous due to low quality, classify as inconclusive and rerun. The value of this case is operational rigor: turning ad hoc “looks good” judgments into reproducible, auditable quality decisions.

Step-by-Step in Genome Forge

1. Open the `Trace/Interop` tab and load the case input record from the training dataset.
2. Run `Trace verify with genotype loci` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/trace-verify`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "verdict": "PASS",
  "genotype_call_count": 4
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case AJ: BLAST-like Similarity Search for Identity, Origin, and Contamination

When an unknown or partially trusted sequence appears, the first practical task is contextualization: what does it most resemble? This case uses local database similarity search to estimate origin and identity. Biologically, strong similarity to an expected construct supports provenance; unexpected top hits can reveal contamination, sample swaps, or untracked design drift. For engineers, think of it as nearest-neighbor retrieval with alignment-based verification rather than metadata-only lookup.

Build a reference panel, run query search, and interpret top hits using identity, coverage, and score together. Avoid single-metric overconfidence: high identity with low coverage can indicate partial overlap rather than true full-length match. Interpret rank gaps between top hits to estimate discrimination confidence. The practical outcome is triage: proceed with expected lineage, investigate mixed identity, or quarantine suspect material. This case strengthens diagnostic reasoning under uncertain provenance.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Run BLAST-like panel search` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/blast-search`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "hit_count": 3,
  "top_identity_pct": 99.2
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case AK: Reference Element Auto-Flagging and siRNA Design/Mapping

Production labs repeatedly screen for known elements: promoters, selection markers, cassettes, and engineered fragments. At the same time, RNA interference workflows require reliable siRNA target design and locus mapping. This case combines both tasks because both depend on robust short-sequence matching with mismatch-aware interpretation. Biologically, auto-flagging accelerates construct sanity checks; siRNA mapping links design intent to transcript targeting feasibility.

Create a reusable reference-element library, scan incoming sequences to auto-annotate known parts, then design and rank siRNA candidates and map chosen candidates back to sequence coordinates. Interpretation should prioritize biological impact: element hits near expected architecture increase confidence, unexpected hits signal contamination or design drift; siRNA candidates should balance GC, repeat burden, and unique mapping behavior. The outcome is a practical decision package for both construct QC and knockdown planning, with explicit uncertainty notes for non-unique or weak candidates.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Reference scan + siRNA design/map` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/reference-scan, /api/sirna-design, /api/sirna-map`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "features_added": 5,
  "sirna_top_score": 92.4
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty

remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Native proprietary `.dna` import is available via optional Biopython parser. Genome Forge DNA containers remain the most reliable interchange format for reproducible collaboration workflows.

Cluster H Cases

Case L: Collaboration, Audit, and Review Governance

Scientific workflows require accountable provenance: who changed what, why, and with what approval. Biologically meaningful claims become stronger when tied to transparent review and role control. This case introduces governance as a quality mechanism.

Create workspace roles, inspect audit trails, and run structured diffs before approval. Interpretation should evaluate completeness of decision trace and unresolved review concerns. The output is not only technical correctness but organizational trust.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Create workspace + submit/approve review` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/workspace-create, /api/review-submit, /api/review-approve`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "review_status": "approved",
  "audit_events": 12
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case T: Batch Reproducibility and Parameter Locking

Reproducibility is fundamental scientific infrastructure. If identical inputs under fixed parameters produce different outputs, confidence in biological conclusions decreases sharply. This case trains deterministic workflow habits.

Run batch analyses with pinned settings and compare against baseline artifacts. Interpretation should quantify drift and classify root-cause hypotheses. Treat unexplained drift as a release blocker.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Batch digest and project history checks` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/batch-digest, /api/project-history-graph`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "batch_count": 6,
  "history_nodes": 9
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case AB: Reproducible Report Package

A result becomes reusable science when accompanied by methods, parameters, context, and evidence summary. Biologically, downstream users need enough detail to evaluate validity and rerun analyses.

Assemble report package with selected outputs, assumptions, and reviewer notes. Interpretation should include confidence statements and unresolved risks. This case teaches documentation as part of scientific method.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Save project + share bundle + load` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/project-save, /api/share-create, /api/share-load`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "share_id": "share_xxxxx",
  "load_ok": true
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case AC: Parameter Sensitivity and Robustness Check

Many bioinformatics conclusions are threshold-dependent. Sensitivity analysis reveals whether conclusions are robust or fragile. Biologically, fragile conclusions should be treated as hypotheses requiring additional data.

Sweep key parameters and track output stability. Interpretation should report stable cores versus boundary-sensitive regions. This strengthens decision quality under uncertainty.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Parameter sweep and compare outputs` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `/api/primer-rank, /api/star-activity-scan`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "stable_window_pct": 82.0
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Case AD: End-to-End Release Checklist and Handoff

Final delivery should integrate design rationale, biological interpretation, validation evidence, and governance artifacts. Biologically, this is the point where computational analysis becomes actionable experimental guidance.

Execute checklist across clusters, confirm evidence completeness, and publish handoff package. Interpretation should identify residual risks and planned follow-up experiments. This case operationalizes professional bioinformatics delivery.

Step-by-Step in Genome Forge

1. Open the `Advanced` tab and load the case input record from the training dataset.
2. Run `Run release checklist tests` using default parameters first, then rerun with one deliberate parameter change.
3. Capture one screenshot of the main visualization/result panel for your lab notebook.
4. Record the API/function used: `smoke_test.py, real_world_functional_test.py`.
5. Save the result summary and classify the outcome as `pass`, `revise`, or `inconclusive`.

Sample Results

Illustrative example values (your exact numbers may differ by dataset and parameters):

```
{
  "smoke_passed": "98/98",
  "functional_passed": "97/97"
}
```

Expected Results

- A concrete output artifact for the case (for example: map, alignment, digest table, trace summary, ranked candidates, or risk report).
- Quantitative indicators that support quality (identity, coverage, mismatch count, GC/Tm ranges, off-target burden, or pass/fail thresholds).
- A short decision-ready status: `pass`, `revise`, or `inconclusive`.

How to Interpret the Results

- Separate *observation* (what the tool reported) from *inference* (what you think it means biologically).
- Check whether key metrics agree. If metrics conflict, treat the result as uncertain and collect another line of evidence.
- Write one recommended next action: proceed, redesign parameters/construct, or repeat experiment with controls.

Biological Explanation

This case links computational outputs to molecular behavior in cells. Sequence changes only matter biologically when they affect functional elements such as coding regions, promoters, splice context, guide/primer binding sites, or structural constraints. Use the case output to explain *why* a molecule is expected to behave differently (or similarly), and what uncertainty remains. Always state limits explicitly (for example, model assumptions, incomplete coverage, noisy traces, or unmodeled cellular context).

Review and audit evidence chain for reproducible delivery.

Capstone

Build a complete bench-planning package using selected cases from all clusters.

1. Design architecture and assay strategy (A-F).
2. Validate construct and evidence quality (G, I, J, Z).
3. Develop intervention plan with risk controls (K, R, V).
4. Run reproducibility and governance gates (L, T, AC, AD).

```
python3 smoke_test.py
python3 real_world_functional_test.py
```

Figure Quality Policy

- Use SVG source files for all diagrams whenever possible.
- Keep figure width constrained to avoid blur in PDF output.
- Prefer vector exports for any replacement graphics.

Glossary

Term	Meaning	CS Analogy
Codon	3-base unit encoding one amino acid	Fixed-width token
PAM	Motif required for CRISPR targetability	Guard condition
Consensus	Best-supported sequence from multiple evidence sources	Weighted merge
Off-target	Unintended match/edit/amplification	False positive
Homology arm	Flanking donor sequence matching target locus	Patch context lines
Chromatogram	Raw fluorescence trace used for Sanger base-calling	Low-level telemetry stream
Genotype Call	Observed base at a biologically relevant position	Assertion on indexed state
BLAST-like Search	Similarity search across a sequence panel using seeds and alignment	Approximate nearest-neighbor plus verification
siRNA	Short RNA used to guide transcript knockdown	Targeted suppression rule
Reference Library	Curated known sequence elements used for rapid matching/ annotation	Signature database